

d make ensure GitLab has the same capabilities (see GitLab Package Registry) |
Do you use build/pipeline artifacts?		Yes	
What is the retention policy of the packages and do you need to migrate them?		365 days, no migration needed	To estimate additional effort for migration or/and advisory service to retain existing ADO feed refactoring application to use GitLab Package Registry
Integrations			
Are there any external tools/applications tied to your ADO server?		Yes. We use an in-house tool that pulls from TFS daily for gathering metrics	If the answer is yes, additional activities will need to be added to the SOW to accommodate transitioning those tools to pull from Git instead.

Other git based SCMs

- We can support these customers by using the "bare git" method of migration. This is done through the Import repo by URL UI or command line using git push -u.
- The customer should provide a list of git urls to iterate over to support the migration.
- Data elements outside of the git envelope (e.g. pull request comments, user membership, etc.) will not be migrated. Only git data elements (e.g. branches, commits, files, tags, etc.) will be migrated.

Non-git based SCMs

- For sources like CSV, ClearCase, SVN, TFVC we can support non automated migration but there is risk that should be factored into these engagements as we dont have deep experience with these source systems. Consider working with a channel partner to find expertise in these source systems.
- Ensure the customer has researched and thought about how their projects will be organized on the destination GitLab system. Often times during the initial transformation from non-git SCM to git scm, there will be splitting of a single trunk of source code to multiple git repositories.

Common customer requests

1. Can we reorganize our projects and groups during migration?

- In congregate, we can do this for source systems of Github Enterprise and Bitbucket. The customer must provide a .csv file with source project url and destination parent path to let us know where the projects will land on the destination system. Also the customer or we must create the group structure prior to a migration with this reorganization involved. There should be additional time added for new group hierarchy creation to the scope.
- We strongly recommend against this for GitLab as the source and destination systems. This seems counterintuitive, but because GitLab has lots of other data outside of the repository structure that sits in the project/group structure that needs to be maintained, the reorganization step should not happen during the migration. If a customer MUST do this, recommend doing it as a pre or post migration step using the UI or API to coordinate the local moves of projects and groups.
- This can be positioned as additional days of migration advisory time. We need to determine what this is a function of to have objective scoping.

2. We're on 13.5 (or some old version), can we still migrate?

- The more of a difference between the source and destination software versions, the higher likelihood there is for data integrity to be lost during migration. The team who maintains the import functionality documented that the importers work when source and destination are no more than 2 minor releases apart from each other. For this reason we include upgrade services to help customers get their source instance to be matching or no less than 2 minor versions from the destination software version. Only in extreme circumstances will we consider exceptions to this guidance.

Other Resources

- See the publicly available Migration Toolkit to help understand the steps taken during delivery of migration services.
- The Customer folder of the migration template project contains useful documentation to help a customer prepare for their migration.
- The Migrating from Self-Managed to SaaS handbook page has details specific to migrating to gitlab.com.

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/migrations/sm-to-saas/index.md

Options for customers migrating from self-managed GitLab to GitLab.com

- Self-service UI migration of groups and projects by direct transfer. Please pay attention that migrating projects by direct transfer is a Beta feature. This is the recommended way to migrate groups and projects between GitLab instances within capabilities of GitLab application. We are working towards reaching General Availability for migrating projects by direct transfer.
- Self-service API based group or project migration by direct transfer.
- Self-service UI based migration using group import/export, however that has been deprecated. Project import/export is not deprecated.
- API based migration using group import/export api and project import/export api
- Professional services automated migration using the Congregate Automation Tool

Why choose Professional Services for migrations to GitLab.com?

PS provides a proven approach that has been, and continues to be, regularly enhanced based on learnings from each customer engagement. Our PS team has experience guiding customers through 100's of migrations to GitLab.com, so we can provide robust guidance on a migration plan that makes sense for a particular customer and what information to communicate to end users to make the experience as smooth as possible for them.

The automated migration tool handles more data than the UI- or API-based approach - things like registry data, CI/CD variables, hooks, integrated K8s cluster configurations, environments, and deploy keys (see Congregate Feature List - GitLab to GitLab for more details). And we migrate all users and their group and project permissions as part of the automated migration. Historical user information regarding the author of MR's, notes, and issues is preserved via the PS migration. This information is preserved as well when migrating groups and projects using direct transfer. It is however not preserved when importing groups and projects by uploading an export files (AKA groups and projects import/export). (See Important Notes section

of project import/export. Note that end users cannot be given administrator permissions on GitLab.com).

Most importantly, GitLab Professional Services Engineers handle much of the migration work asynchronously, freeing up the customer's administrators to focus on other valuable tasks instead of needing to make the GitLab migration an initiative that requires all of their effort.

How much does a Professional Services migration from self-managed to GitLab.com cost?

The total price of each engagement varies based on the size and complexity of the migration. While not a perfect formula, we currently estimate the scope of the migration based on the number of users and projects in the self-managed system. Note that GitLab repos > 5 GB cannot be migrated via our automated tooling and will significantly increase the scope of a self-managed to GitLab.com migration. See Why are Large Repos so Expensive to Migrate in the FAQ at the end of this page for more information. Also, please note that the source self-managed instance of GitLab must be within two minor versions of the last released version, or else an upgrade will be needed. That can be included in the final estimate as well.

As a general guide, we've provided rough order magnitude (ROM) estimates below for migrations under standard conditions. Please note that each customer migration needs to be scoped individually to validate assumptions in the ROM table. Upon confirmation that the customer would like to proceed with Professional Services, we will generate an SOW that will need to be signed by the customer.

ROM Estimates

Price in each cell represents a rough order of magnitude based on number of users and projects under 5 GB. Add an additional \$12k for every 10 Git repos > 5 GB.

Users (row) / Projects (column)		1,000 or less	1,001 - 2,000	2,001 - 3,000	3,001 - 4,000	4,001 - 5,000	5,001 - 6,000	6,001 - 7,000	7,001 - 8,000	8,001 - 9,000	9,001 - 10,000
--- --- --- --- --- --- --- --- --- --- --- ---		500 or less	\$30-40k	\$50-60k	\$70-80k	\$90-100k	\$100-110k	\$120-130k	\$140-150k	\$150-160k	\$170-180k
501 - 1,000		\$35-45k	\$55-65k	\$75-85k	\$90-100k	\$105-115k	\$120-130k	\$140-150k	\$155-165k	\$175-185k	\$195-205k
1001 - 2000		\$50-60k	\$65-75k	\$85-95k	\$100-110k	\$120-130k	\$135-145k	\$155-165k	\$170-180k	\$190-200k	\$205-215k
2001 - 3000		\$60-70k	\$75-85k	\$90-100k	\$110-120k	\$125-135k	\$145-155k	\$160-170k	\$175-185k	\$195-205k	\$210-220k

If you have more than 10,000 projects or 3000 users, please reach out to the Engagement Managers and they will work with you to provide a ROM estimate.

My customer wants to proceed with PS for migrating to GitLab.com - what is the next step?

After you've had initial conversations, if your customer wants to proceed to further explore PS for migrating to GitLab.com, please initiate a scoping request via the Services Calculator.

Enter the customer name, your GitLab username, walk through a quick questionnaire to provide some additional information, and click the Create PS Scoping Issue button.

There will be a <Customer Name> - Scope Issue and Write SOW issue created by the calculator, as a child of an epic with the name of the customer. That issue will have the information we need for scoping the customer engagement pre-populated as a table in the description (with the heading SSOT for "single source of truth"). Please fill in the appropriate fields in that table - bold font fields are required in order for us to scope the migration effort.

Once you have provided the required scoping info, an Engagement Manager will generate an Engagement Estimate, using standard formulas we have created for scoping. We will produce a Google sheet that you can pdf and use for discussions with your customer, either async via email or sync via a live meeting. The Engagement Manager would be happy to support a live meeting, schedule permitting, to review the information with the customer. You can see a sample Engagement Estimate with descriptions of each activities.

If the customer agrees to the estimate, and a conversation validates that the information used in scoping is accurate for the customer's situation, then the final step will be for the Engagement Manager to create an SOW that will be approved internally, and then you can send that to the customer for signature. See Selling Professional Services for additional details on this process.

Gathering data to help scope a migration

There are a few questions GitLab uses to assist in the SM to GitLab.com transition process. Two of those questions require a Customer Admin to provide data to the GitLab team:

- 1. How many GL repos are larger than 5GB approximately?
- 2. What is the typical container/package registry size of your projects?

GitLab provides a tool to gather data about the existing environment. Download and run the Evaluate tool with an admin token and provide all the generated files back to the account team.

Additional Selling Tools

- See the WIP - Selling Professional Services - SM to GitLab.com Google slides presentation

FAQ

1. Why are large repos so "expensive" to migrate? The GitLab.com instance has a hard limit Cloudflare 5GB file import/git push. Any repo > 5 GB must be migrated manually via workarounds to remove large files and, in some cases, break the import into smaller pieces. These imports require significant effort so we can only manage 2 repo imports per day.

1. What can my customer do to make the migration easier and potentially reduce the PS engagement cost?

- Since large repos greatly increase the scope of PS efforts, customers should follow the guidance provided in the Reduce Repository Size documentation to clean up their repositories prior to migration.

- The migration depends on users in the source and destination system having the same primary email address, so it is helpful to review user accounts in the primary system ahead of time, especially if the customer plans to utilize SSO with SCIM provisioning on GitLab.com.

- Groups and projects are migrated in the same hierarchical structure as they are defined in your self-managed system, so if the customer wants to restructure groups or projects, they might want to undertake that before the migration.

- On GitLab.com, public projects are available to the entire internet, which might be a big shift for a customer using self-managed GitLab behind their firewall. Additionally, there are no internal projects on GitLab.com. So if the customer has projects with these access settings, they will want to review them and start thinking about how to shift project accessibility on GitLab.com.

Our Professional Services team can incorporate Advisory Services into a migration SOW to help with any of the above activities if the customer needs additional guidance.

1. Can partners perform SM to GitLab.com migrations? Partners are limited to the same restrictions as customers - they cannot have administrator permissions on the GitLab.com platform, and they must utilize known ui or api based migration approaches with their own scripted solutions. GitLab partners can provide guidance on a migration plan, in terms of considerations for which groups and projects to migrate when, and what activities need to be performed, and what information needs to be communicated, to end users as their projects are migrated. But, partner migrations have limitations, such as:

- MR, notes, and issue authors are not preserved on migration
- Git repos > 5 GB can be migrated using the documented workaround

We sometimes pair with a partner so that they do some of the advisory work and we run the automated migrations on their behalf. Or, if a customer is not concerned about maintaining authors of MR's, notes, and issues, then the partner can perform the migrations themselves. If the partner needs guidance in cleaning up large repos for import, they can subcontract that part of the migration to us.

SECTION: customer-success/professional-services-engineering/examples/partner_invoice_mockup.md

Partner Company Name INVOICE

INVOICE 101

DATE: NOVEMBER 26, 2024

PO: 10003403404

TO:

md
GitLab
ap@gitlab.com
268 Bush Street, 350
San Francisco, CA 94104

DESCRIPTION	QUANTITY	UNIT PRICE	TOTAL
-----	-----	-----	-----

Kantata Project Name, November 1-30, 2024, Resource Name	30	\$100	\$3,000.00	
Kantata Project Name, November 1-30, 2024, Resource Name	20	\$100	\$2,000.00	
SUBTOTAL		\$5,000.00		
SALES TAX				
SHIPPING & HANDLING				
TOTAL DUE		\$5,000.00		

Make all checks payable to \[Company Name\]

If you have any questions concerning this invoice, contact \[Name, phone, email\]

THANK YOU FOR YOUR BUSINESS

SECTION: customer-success/professional-services-engineering/positioning/index.md

Where to position PS in the Customer Journey

Most professional service engagements happen during the onboarding, implementation, and adoption phases of the customer journey.

When opportunities are approaching closed/won and planning for these stages is occurring, its a good idea to consider bringing in professional services to help expedite the customer's transition to using GitLab to enable them to realize value faster.

This page aims to help the ISR/SAE understand the services that certain types of customers typically engage in and how to position those services when building out a success plan.

Value of GitLab Professional Services

<!-- Elevator Pitch

The Problem - Customer Perspective

The customer has decided that GitLab is their tool of choice going forward, but has concerns about realizing the value of GitLab quickly and seamlessly. These concerns can be in various areas:

- Lack of subject matter expertise on the GitLab technology stack
- Lack of adequate resources to promptly stand up GitLab
- Lack of sufficient resources to stand up GitLab in a highly-available or geographically dispersed way
- Concern around a large number of users coming off of legacy systems
- Concerns about change management for new processes
- Lack of confidence in the ability to deliver on the transformative promise of adopting "all"

of GitLab

Solution -->

Single Sentence

With PS engagement, customers start leveraging the full capabilities of GitLab earlier with improved proficiency, reduced risk, and increased competitive advantage.

Short Message

"The whole is greater than the sum of its parts" - and this is particularly true in modern software development. GitLab makes the collaboration around software delivery as efficient as possible. We provide services to minimize the transition period to GitLab and help you get back to peak efficiency quickly. We provide specialized training to educate users in these software development and delivery practices using gitlab.

Long Message

Adopting GitLab gives you the only single DevOps platform on the market. GitLab Professional Services will help you align your people and processes to match.

Our Professional Services team is made up of not only GitLab subject matter experts but seasoned DevOps professionals who have experience in deploying and maintaining both large-scale applications as well as creating and teaching best practices throughout the SDLC. Our experts help lead DevOps Transformations, providing direct support to our customer's strategic business initiatives. Their experience with other Fortune 500 enterprises allows you to crowd-source your enterprise's digital transformation.

GitLab's Professional Services team exists to enable you to realize the full value of your GitLab installation. We can provide direct implementation support to ensure your GitLab installation is resilient and secure. We also offer migration services to facilitate your transition to GitLab by delivering a clean dataset to resume operations at once. Our education and specialized training provide training in best practices, such as CI/CD, version control, metrics, and more.

Customer Archetypes

Mid-Market to Enterprise coming from non-gitlab SCM

These customers are adopting GitLab usually for a number of reasons, but typically use SCM as their entry point. They have source code management happening in their company but it might not be standardized into a single process or tool. This means they will need to migrate data and users to (sometimes git and) gitlab.

The larger the company, the higher likelihood they will be opting for GitLab self-managed. If this is the case, they will need to deploy GitLab typically with aspirational requirements of system High Availability, Disaster Recovery and performance and might not have the technical expertise in their current staff.

They have a team that manages the current tooling at the highest level, and are often competing against shadow IT from setting up their own (less restrictive) instances of SDLC tools. This team has probably never managed a large migration and needs help with the rollout plan. Their engineering teams (end users of gitlab) are on the continuum of knowledge about latest git and GitLab best practices.

To get to first value as soon as possible, you should consider:

- Implementation (infrastructure deployment and GitLab configuration)
- Migration (SCM Data and user migration)
- Education (Basics, CI/CD and Admin)

SMB to Mid-Market moving to gitlab.com

These customers typically move to GitLab for the benefits of an all-in-one SDLC platform to reduce toolchain tax.

They will likely have a small team responsible for transformation that will be managing this transition. Its unlikely they have the expertise or time to manage a data and user migration without slowing down their business significantly. They will also probably want to integrate with other systems like LDAP, Jira, or Jenkins. Finally, their users will probably need upskilling on the basic and advanced features of GitLab to ensure they're getting the most value out of the gitlab.

To get your customer the most value as soon as possible, you should consider:

- Migration (SCM and/or CI/CD based on entry point)
- Integration (LDAP/SSO, Jira, Jenkins)
- Education (Basics and/or CI/CD)

SMB or Mid-Market without migration needs

Some customers have a team of git ninjas who can manage migration and setup quickly, but the rest of the engineers might not be as skilled. Its always a good idea to suggest education services because the customer end users will be more likely to push for later stage adoption. This type of grassroots motivation will go a long way when investigating conversion opportunities.

For these customers consider our Education Services

Sales Collateral

Internal Testimonials

The PS team has been building maturity and repeatability to its services over the later part of 2019 and early part of 2020. We have captured some recent wins with internal testimonials in this highspot page

Pitch Deck

To discuss our services offerings with prospects, it is often helpful to have a few slides to describe the role of the professional services team. If you need slides for our SKU services feel free to pull from the below deck. If you are thinking this is a larger engagement, please contact your Engagement Manager

Professional Services Proposal Deck

Data Sheets

Professional Services Data Sheets are available as subpages to the marketing site. You can find them linked on the catalog page.

Other Collateral

- How to sell professional services
- General Guidelines for working with professional services

SECTION: customer-success/professional-services-engineering/processes/delivery-checklists.md

Overview

To alleviate potential last-minute issues concerning instructor preparation, the following pre-training checklist is provided below as a reference guide to ensure that the instructor is fully prepared. Additionally, a post-training checklist can be useful for ensuring that the instructor has provided feedback to the appropriate GitLab team members and to triage any parking lot questions or items that arise during the training class.

Pre-Training Checklist

The pre-training checklist below provides a list of tasks and reminders to help you fully prepare for your upcoming training. The activities should be completed prior to the first day of class.

- Ensure that you have training logistics information, such as training dates, times, and main customer point of contact.
- For onsite training, make sure that you have the training venue address, point of contact information, sign-in procedures and required documentation, if applicable.
- Make sure to review all of the trainer materials, such as the slide deck, hands-on guide, facilitator guide, and GitLab documentation for the course you are delivering.
- Please ensure that you use the latest version of the slide deck for your upcoming class.
- Ensure that the lab environment and lab exercises are working properly by reviewing ahead of the first day of class.
- Ensure that you can access and have full functionality of the teleconferencing system.
- Review the Remote Training Facilitation Best Practices Tips.
- Declutter by closing unneeded tabs in the browser you will use to show the course slides.
- Silence or quit Slack and email to prevent notifications during your class.
- Set up your laptop's environment in whatever ways are necessary for the class. For example, if you're going to install a GitLab Runner as part of a CI/CD class, make sure you've

uninstalled the GitLab Runner binary from your laptop.

Post-Training Checklist

The checklist below provides a list of tasks and reminders to help you post-training delivery. The activities should be completed upon training delivery.

- The Education Services team is available via your respective GitLab Slack channel for any immediate concerns during your training.
- Research and answer any parking lot questions that came up during the training.
- For course slide deck or lab instructions feedback, concrete suggestions are best, but all suggestions are welcome.
- Provide any follow up information or feedback to the Education Services team to be forwarded to the GitLab Account Team.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/_index.md

- What is the Professional Services Delivery Methodology (PSDM)
- Managing a Project in GitLab
- Project Velocity and Iteration Scheduling
- Status Planning
- Status and Burndown Reporting
- Mitigating Risk via RAID Board
- Internal Retrospective Guidelines
- Customer Retrospective Guidelines
- Guidelines for Iteration based PSDM

What is the Professional Services Delivery Methodology (PSDM)

The Professional Services Delivery Methodology (PSDM) is the guiding framework for Program and Project Delivery within GitLab. The goal is to ensure the PS Delivery team operates within a predictable timeframe against the Project scope while prioritizing Customer Success. This is achieved through:

- Using GitLab.com as the Single Source of Truth (SSOT)
- Following label guidelines for managing and reporting progress
- Identifying and mitigating risks
- Actioning/tracking improvements internally
- Implementing iterations based for Sprint/Iteration-based SOWs
- Utilizing enablement materials
- Applying fundamental Agile best practices

Managing a Project in GitLab

GitLab serves as both a project management and collaboration platform for all Professional Services engagements. We utilize the following features/terminology within GitLab for project management:

Please reference the GitLab best practices when navigating GitLab.

For initial configuration of GitLab as a Project Management tool, see the configuration guide.

> NOTE: Any issues marked as "internal" are still visible to anyone who has "developer" access into the GitLab Collaboration project. This includes people outside of GitLab. It is recommended to use the Project's "Internal Epic" for confidential communications.

Project Management Mapping in GitLab

PM Term	GitLab Definition
-----	-----
Group	This is the landing page for the Customer project and serves as the single source of truth for Project Governance.
Projects	If an engagement has more than one Workstream (or SOW), we will track these using multiple Projects. Change Order activities will be tracked against the original project (SOW).
Boards	Typically organized by labels or scoped labels to keep the team on track together. In some cases, the GitLab/Customer team may utilize multiple Boards across Projects.
Epics	Generally derived from Workstreams or "Activities" outlined in the SOW.
Issues	These are the atomic units of work that roll up into an Epic.
Iterations	Time-boxed (generally two-week) events that are reviewed during Agile ceremonies.
Milestones	Used to track progress against Project Phases
Labels	Used in various ways, with the most important applications being: - Managing progress during delivery using a left-to-right workflow - Managing prioritization - Organizing specific sub-categories of work - Tracking and mitigating risks
Weight	Indicates the size or level of effort of an issue. See Good Estimation Techniques for guidance on assigning weight

For additional clarity on mapping Agile terminology to GitLab, reference this guide.

Label Guidelines

Labels are the most effective way to generate reports around Projects and organize information according to the Project team's needs. While teams have flexibility to create labels that meet their specific project reporting requirements, there are established guidelines for label generation for internal use.

Our CP (Customer Project) automation includes the following default labels:

- SOW- or PO - helps the GitLab team search for Projects within the Professional service Group
- PM name - helps the GitLab team sort by PM name
- PSD workflow (for issue board management)

Labels used for Internal retrospectives & RAID tracking/reporting can be found in the "Reporting throughout the Iteration" section below.

Project Velocity and Iteration Scheduling

The iteration schedule and cadence is first introduced in Iteration 0 and becomes part of the Engagement Charter within the GitLab Customer Project (Group). It is crucial that the Customer agrees to an Iteration Schedule as an output of the Customer Kickoff. However, this should be initially introduced and collaboratively developed with the Customer during the Stakeholder Planning meeting.

When working within an Iteration/Sprint schedule, there are five key components:

1. Iteration Planning - Setting goals and selecting work items for the iteration
2. Daily Standup - Brief synchronization meetings to share progress and identify blockers
3. Backlog Refinement - Clarifying, estimating, and prioritizing upcoming work
4. Iteration Review - Demonstrating completed work and collecting feedback
5. Retrospective - Reflecting on the process and identifying improvements

When not working within Iterations, it's still important to clearly communicate the project velocity to ensure consistent resource alignment. For example: "We anticipate working 40 hours/week for 5 weeks to perform a successful upgrade and seamless migration of 1000 users and 2000 projects from GitLab SaaS to Self-Managed for FedRAMP compliance." This is validated during the Stakeholder Planning meeting and Customer Kickoff.

Status Planning

For Agile SOWs, the Program Manager / Project Manager provides strategic direction for the project, including the vision, product roadmap, release goals, and iteration objectives. The Program Manager / Project Manager is responsible for managing the product backlog by inserting, re-prioritizing, refining, or removing items as needed. This can occur at any time until the iteration scope is defined and committed to by the development team.

Please reference Backlog Management for guidance on estimation, backlog grooming, and preparation for Iteration Planning.

If the scope of work is more strictly defined within the SOW, the PM is expected to assess the impact of any changes and work with the Customer to either reprioritize or initiate the Change Order process.

Status and Burndown Reporting

Who Updates What?

While the PM is responsible for preparing ceremonies, reporting status, and managing Issues, the GitLab PSE (Professional Services Engineer), TA (Technical Architect), and Customer team members are all expected to contribute to and update their assigned issues (tasks) within the Project board. This highlights the importance of establishing effective Working Agreements during Iteration 0.

Working asynchronously and remotely presents unique challenges. Ensuring that the Directly Responsible Individual (DRI) actively contributes to their assigned issues is vital to maintaining project velocity. This approach allows the PM to shield technical teams from

distractions while enabling effective status roll-up.

Not every Project will follow a Sprint or Iteration cadence. However, status updates should be documented within GitLab and reported to the Customer on a weekly basis. It is recommended to use an Engagement Charter to direct stakeholders to weekly status updates, RAID logs, and other relevant project documentation.

Status Report Examples:

- Engagement Charter Example
- Status Update Example 1
- Status Update Example 2
- While the GitLab CP is the recommended platform for status reporting, if the Customer prefers a presentation format, please reference this template.

Project Burndown

While Kantata is the single source of truth for our Financial Management, we recommend PMs track the hourly and financial burn separately for additional auditing and progress against the progression of the Project. Templates can be found [here](#). It is required to track hourly/financial progress within the weekly status report

Mitigating Risk via RAID Board

The RAID (Risks, Actions, Issues, and Decisions) framework serves as our single source of truth for project risk identification, management, and resolution. It provides several key benefits:

- Creates transparency for project stakeholders and leadership
- Enables proactive monitoring of projects with Yellow/Red health status
- Centralizes risk documentation and mitigation strategies

Setting Up RAID

- The RAID board is automatically created when the PM initializes the CP template
- Rename the template to "RAID - [Customer Name] - [SOW/PO]"
- While the PM is responsible for creating and managing the RAID, both internal team members and Customer representatives are encouraged to update it as challenges and mitigations evolve
- [View an Example RAID Log](#)

Reporting on Risk via RAID

When documenting a risk, follow these guidelines:

1. Apply appropriate labels:
 - "PS RAID Log"
 - "PS Risk"
 - "PSD Workflow::Not Started"
2. Define severity using priority labels:
 - "PS:: Priority High"

- "PS:: Priority Medium"
- "PS:: Priority Low"

3. Quantify impact in terms of:

- Timeline effects
- Scope changes
- Quality implications
- Budget consequences

4. Assign ownership:

- Designate a person responsible for accepting, mitigating, or resolving the Risk

5. Detail mitigation plan:

- Document specific actions to lessen or eliminate the Risk

6. Escalation process:

- Use the "Escalated" label for risks requiring immediate attention that impact project progress
- Escalated items should be surfaced in the Professional Services Portfolio report

Internal Retrospective Guidelines

The Internal Retrospective issue serves as an ongoing reflection mechanism throughout the project lifecycle, focusing on:

- Capturing team accomplishments and celebrations
- Documenting technical challenges and solutions
- Identifying process improvements
- Recording business development opportunities
- Building institutional knowledge for future engagements

Unlike the Customer Retrospective which occurs at project milestones and reviewed with the Customer, the Internal Retrospective serves as a way to capture improvements needed to improve our processes and celebrate team wins

Setting Up the Internal Retrospective

The Internal Retrospective is automatically created when the Customer EPIC is established:

1. Location:

- The issue lives within the internal Customer EPIC
- It should be linked in the internal Project Slack channel for easy access

2. Template:

- Use the latest template to ensure standardization
- The PM should ensure the template is properly populated with project information

3. Access:

- This is an internal-only issue - no customer access should be granted
- All GitLab team members working on the project should have access

Contributing to the Internal Retrospective

Team members should contribute to the Internal Retrospective throughout the project:

What to Document

1. Project Wins and Celebrations:
 - Technical achievements
 - Customer praise and positive feedback
 - Team collaboration successes
 - Innovative solutions
2. Lessons Learned:
 - Technical challenges and how they were overcome
 - Process gaps and workarounds
 - Communication improvements
 - Tools and techniques that proved effective
3. Opportunities for Improvement:
 - Areas where processes could be streamlined
 - Documentation gaps
 - Training needs identified
 - Tool limitations
4. Assets Created:
 - Scripts and code snippets
 - Documentation templates
 - Configuration guidelines
 - Customer-specific solutions with reuse potential
 - Any Delivery Kit related updates

How to Contribute

- Ongoing Basis: Team members should add comments to the issue as noteworthy events occur
- Weekly Check: The PM should encourage updates during weekly team meetings
- Phase Transitions: Special attention should be given to capturing learnings at the end of each project phase

Finalizing the Internal Retrospective

At project completion, the PM should:

1. Conduct a Final Session:
 - Schedule a 30-60 minute Gitlab team retrospective call, inviting PS Practice team members
 - Review contributions and synthesize key learnings
 - Identify patterns and major takeaways
2. Categorize Learnings:
 - Apply appropriate labels to facilitate future searching:

3. Create Follow-up Issues:

- Generate specific issues for action items
- Assign owners for implementation
- Set realistic timelines for completion

An example Internal Retrospective can be found [here](#). The list of inflight and completed internal retros can be found in the "PS-Plan" Gitlab Project by searching within "Titles" and the text "Internal Retro".

Knowledge Sharing

PS leadership regularly reviews retrospective data to:

- Identify common challenges across projects
- Recognize exceptional team performance
- Inform training and enablement priorities
- Update delivery kits and methodologies
- Shape future service offerings

Best Practices

- Keep entries objective and solution-focused
- Include specific examples rather than generalizations
- Maintain a balance of positive observations and improvement opportunities
- Focus on systems and processes rather than individuals
- Quantify impact where possible (time saved, problems avoided, etc.)
- Suggest specific, actionable improvements

Customer Retrospective Guidelines

The Customer Retrospective is a critical component of the Professional Services Delivery Methodology that provides structured feedback on the engagement alongside our Customers. It serves to:

- Capture the customer's perspective on the engagement
- Identify what went well and areas for improvement
- Document successes and challenges for future reference
- Strengthen the customer relationship through collaborative reflection
- Generate insights that can inform future engagements
- Encourage CSAT participation

Setting Up the Customer Retrospective

The Customer Retrospective issue is automatically created when setting up the collaboration project:

1. Initial Setup:

- Rename the template to "[Customer Name], [SOW/PO]" when setting up the project CPR
- The issue serves as a living document that both GitLab and Customer teams can contribute to throughout the engagement

2. Ongoing Documentation:

- Much like the internal retrospective, encourage Customers to document feedback throughout the engagement
- Regular updates help capture insights while they're fresh and relevant

Conducting the Customer Retrospective

Preparation

1. Schedule the session:

- Plan a closure/retrospective meeting with the customer at the end of the PS engagement
- Share the retrospective issue in the meeting agenda for team members to pre-fill before the session

2. Encourage participation:

- Ask team members to add their initials before their feedback under each main section
- If someone agrees with existing feedback, they should add their initials alongside the original contributor

During the Session

The retrospective meeting should cover these key sections which are found in the issue template:

1. Attendees and Date
2. Summary of Services & Value Delivered
3. Final Burn Down Report
4. Retrospective Discussion
 - What went well
 - What could be improved
 - Lessons Learned
5. Feedback on PS processes or Methodology
6. Voice of the Customer

Documenting and Following Up

After the retrospective session:

1. Finalize the document
2. Apply appropriate labels
3. Create action items
4. Share insights within ps-internal

Best Practices for Customer Retrospectives

- Keep the tone collaborative and constructive
- Focus on specific, actionable feedback
- Balance discussion of challenges with celebration of accomplishments
- Capture both technical and process-related insights
- Ensure customer's voice is prominently featured

- Document quotes and testimonials (with appropriate permissions)
- Use the retrospective to strengthen the customer relationship

Additional Resources

- Retrospective Facilitation Issue/Guide
- Customer Survey Template

Guidelines for Iteration based PSDM

The full PSDM implementation with a complete Iteration schedule is required only when the Agile-specific SOW

- Exceeds 5 Iterations, or
- The engagement is planned to exceed two months

Please review the archetype definitions to understand what constitutes a "large" Customer engagement.

Use the following guide when planning for Iteration 0:

	Projects > 5 Iterations (~9 weeks, incl. iteration 0)	Projects < 5 Iterations
Managed in GitLab as SSOT	Required	Required
Iteration 0	Required	Required
Internal Retrospective tracking	Required	Required
RAID tracking	Required	Required
Backlog Refinement	Required	Only as needed
Daily Standup	Required (with customer-determined cadence)	Only if needed
Communication Plan	Required	Required
Iteration Planning	Required	Only as needed
Iteration Review (status report)	Required	Weekly Status at minimum
In-project Retrospectives	Required	Only if needed

<!--

What Kind of Engagements is PSDM Suited For?

PSDM is intended to be used on the large, complex, customers (see archetype definition) that are falling into one of two use case scenarios:

1. Large Scale Tool Adoption - scaling usage of GitLab across the organization (tool adoption use case), or are
2. DevSecOps Transformation - assisting customers with transformational activities (transformational use case)

Some characteristics for both use cases include:

Large customers (Fortune 500)

Committed to substantial investments in new technologies and personnel, as well as a deep commitment to change at all levels of the organization

Considering tools adoption and digital transformation a strategic overhaul that seeks to maximize digital tools to improve performance, meet customer expectations, and innovate
Trying to shorten cycle time, and increase time to market velocity
Involving changes that impact people, process, and technology
Have been sold on GitLab benefits
One platform
Simplification of the DIY toolchain and saving on license costs
Shift-left security
Enhanced developer experience

The difference between the tool adoption use case and the transformational use case lies in the complexity and breadth of the approach.

Tool adoption is frequently limited to a set number of users, departments, and work streams, making the application of the methodology more predictable. Tool adoption frequently are multi-workstream engagements that manage parallel efforts of SCM migration, CI/CD transition, Security tool consolidation and workforce enablement and training. This use case is similar to traditional IT system rollouts:

1. Install GitLab SM / configure SasS / configure Dedicated
2. Migrate source repositories and user data - in waves
3. Train user in groups
4. CI/CD transition - DevSecOps Workshop, CI/CD Workshop, DevSecOps App Transformation, CI/CD App Modernization
5. Security tool consolidation
6. Training and Education

Transformational activities frequently involve cross departmental analysis of cycle times, processes, technical architectures and tying specific technical activities to desired business outcomes (see a simple definition for Digital Transformation [here](#)). Transformations frequently address business goals and technology adoption and adopting new ways of working in order to achieve step change improvements in business performance. This necessitates transformational activities to be focused on:

1. People
2. Process
3. Technology
4. Continuous Improvement, and
5. Key Performance Indicators - DORA metrics

Key Principles

A Word about Terminology

Just like GitLab in general, Professional Services follows a light-weight, iterative, process. The goal is to be efficient and effective while minimizing administrative burden.

Most agile, iterative, process methodologies (such as Scrum, SAFe, or PMI-DA), define a set of terms and conventions. So does GitLab, and there is a simple mapping of agile to GitLab terminology that documents the differences while acknowledging that essentially their meaning

is the same.

Because industry parlance and GitLab terminology only differ slightly (for example, the common industry standard term for a time-boxed development cycle is "sprint" or "iteration", whereas GitLab uses the term "milestone") PSDM uses some of the terminology interchangeably.

Principles

There are eight (8) key principles to make any GitLab engagement successful. These are generally acknowledged to be industry best practice, and customers that understand and fully embrace these principles are more likely to succeed. This is important for PS to stress with customers before and during engagements.

The key principles are:

1. Optimize communication · especially focused on executive, buyer, and stakeholder alignment and addressing progress towards business outcomes
2. We iterate and work in Small Batches - with work conducted according to a cadence and organized in backlogs and iterations / sprints
3. Enable ownership and personal accountability
4. Small Teams equal better and faster communication
5. Localized decision making · not in terms of geography but team organization
6. Allow for continuous improvement / continuous learning
7. Understand the customers' products, value streams and associated lead (wait) times - critical to optimizing cycle times and achieving business goals
8. Embrace a new way of thinking, planning, and budgeting - instead of waterfall upfront scheduling and budgeting, work through iterative, prioritized deliverables, against a fixed project budget

These principles underly a disciplined 11 step implementation approach.

11 Steps to Success

It is important to point out that GitLab's PSDM is agnostic to popular implementation and process methodologies and can seamlessly be applied regardless of what the specific customer circumstances are. The steps are:

1. Identify, document, and conduct an engagement strategy based on the specific customer situation
2. Build awareness and excitement at all levels - regularly communicate status to all relevant parties: executives, buyers, stakeholders
3. Identify MVP pilot project(s) - we always utilize one or more MVPs to prove out our engagement strategy
4. Train the customer development team(s)
5. Develop product backlog and estimates for the engagement
6. Run iterations / sprints producing incremental value
7. Identify metrics - make data centric decisions
8. Gather feedback and improve · adjust the process
9. Mature
10. Scale across other teams, programs, portfolios - scale in waves

11. Regularly Assess and Refine

It is recommended to encourage the customer to follow GitLab Best Practices.

Iterative Basics

1. Harden Implementation Success in MVP Pilot(s)

Each engagement will successfully deliver one or several MVP pilot(s) - depending on the size and complexity of the engagement.

MVP Pilot Phases of an engagement consist of:

1. Discovery - The initial phase to assess capabilities and validate assumptions to ensure we're building the right thing and meeting customer needs
2. Team Readiness - The process to define people, process and structures to prepare teams operating in an iterative way
3. Sprint 0 - The initial sprint where the team aligns on norms and objectives, and key technology choices
4. Sprinting - The execution and delivery process for teams to deliver the GitLab engagement

Scaling up across the organization is achieved by breaking down the overall implementation effort into waves that will be delivered successively - after successful delivery of one or several MVP pilot(s).

2. Integrate Business and IT

Integrating business and IT needs is accomplished through a disciplined decomposition of higher level Vision and Goals into Epics and User Stories, which in turn drive the technical implementation of the customer engagement. Depending on the size and complexity of the engagement, one or several Program Managers are closely coordinating with executive level stakeholders and business sponsors in order to ensure that key business goals are achieved. Lower level technical coordination and tasking is done by one or several projects managers, working with the GitLab TA/PSE Team and the customer sourced Dev Team.

3. Adopt GitLab's Consistent Tools

Helping customers adopt the GitLab platform across the entire software development life cycle delivery on two key promises:

1. Achieving significant license savings by replacing a complex DIY tool chain
2. Enabling productivity improvements due to an enhanced developer experience, streamlined collaboration, and reduced context switching

4. Optimize Internal Controls

By optimizing and automating internal controls (sign-offs / approvals), cycle times are significantly reduced and Time to Market windows are shortened. Collaboration is streamlined, context switching is eliminated, and velocity is increased. Overall productivity increases.

5. Establish Thought Leadership through Continuous Value Delivery and Cost Savings

GitLab consultants are ·catalysts· who show direction and provide coaching

Focus is on setting up systems for people to work effectively

Small, autonomous cross-functional teams deliver value constantly using GitLab, we live our values

Teams are empowered to make decisions and are responsible for end-to-end outcomes

Our engagements enable customers to conduct production deployments automatically without lengthy approval processes

Cycle times are greatly reduced and value streams are more optimized

Cost savings are achieved due to a simplified tool chain using the GitLab platform -->

<!-- PS Process & Methodology Mapped to the Customer Journey

The Professional Services process and methodology fits within the Customer journey that is supported by Customer Success. Professional Services contributes to the customer journey from the point of SOW Close through the Project Close phase.

](<PS Delivery Customer Journey Flow - Page 1 (11).jpg>)

Source, GitLab Team Members Only

PS Process Methodology Stages

The above diagram (slide 4) is meant to describe the Directly Responsible Individuals (DRIs), Activities, Outcomes, and Tools/Collateral for each stage of the methodology. We can also see clear categorization of stages in pre-sales and post-sales phases.

In the linked pages below, you can see a detailed drill down into the steps within each stage that individuals use to perform activities to deliver desired outcomes per each stage. These pages are split by the Phase of the selling process (Pre-sales vs Post-sales).

Scoping (Pre-Sales)

For a more detailed explanation of the steps that comprise each stage of the scoping phase, check out the Scoping (Pre-Sales) page. In this page, we drill down into- and describe- specific steps in each phase of the pre-sales scoping process.

Delivery (Post-Sales)

For a more detailed explanation of the steps that comprise each stage of the delivery phase, check out the Delivery (Post-Sales) page. In this page, we drill down into- and describe-

specific steps in each phase of the post-sales delivery process.

-->

<!--

Overview

The purpose of this page is to document the sales-assisted selling motion used by Professional Services Engagement Managers and Regional Delivery Managers. If you're on a GitLab sales account team looking for information, try these pages on positioning professional services or selling professional services.

This page will help outline the when and how to get involved with positioning and scoping services, how to estimate, how to use SOW generation software, and the processes to gain approval.

> Note: Services engagements can take two forms. This will focus on the custom SOW scoping process, not the Standard SKU process.

For custom SOWs, the workflow for SOW creation involves a partnership between the Account Team and the Professional Services Team.

Source (GitLab Team Members Only)

1. Positioning

- DRI: Account Team (SAE/AE, SA, CSM)
- Supported By: Engagement Manager

MEDDPICC

Professional services can be positioned when a prospect becomes a customer (e.g. the Land of a new deal) or when an already existing customer is growing their staff or interesting in adopting new features of GitLab (e.g. expansion). The SA, SAE, AE, or CSM (e.g. "Account Team") is primarily responsible for this process, following the Command of the Message and MEDDPICC messaging frameworks.

For the larger, more strategic customers PS Engagement Managers tend to get involved earlier in the selling process to help with discovery and provide lessons learned on rollout from past engagements. For the medium sized customers, Engagement managers tend to get involved with account teams when the SFDC stage 4 (Proposal) is achieved.

Customer Success Discovery

The Account team should be encouraged to use the Customer Terrain Mapping templates to help the customer to start thinking about how they will achieve their longer term success. This typically results in conversations about services to aide them in their journey.

Services Needed

Once its identified that the customer will likely want to engage with professional services, its the responsibility of the Account Team to get in touch with the Engagement Manager.

2. Scoping

- DRI: Engagement Manager
- Supported By: PS Practice, Account Team (SAE/AE, SA, CSM)

Gather Data

After the services calculator is run by the Account Team, scoping issues and Project Scheduling Intake issues are automatically created and land in the PS Plan Project. Using this customer scoping issue and Project Scheduling Intake issue, the engagement manager gathers data asynchronously from the account team. Questions about the potential engagement can sometimes be answered by the Account Team from the discovery that was done already. We want to make sure we avoid asking duplicate questions to the customer. In some cases when the account team cannot provide the level of detail to create an egagement, the EM will meet with the Account team and Customer to ask additional discovery questions to get to a level of detail needed. Once the data has been gathered, the EM will populate the answers to the Project Scheduling Intake issue.

Create Engagement Estimate

A PS Engagement Estimate spreadsheet used for scoping services. The Engagement Manager uses the PS engagement estimate spreadsheet to define the services in scope and estimate the amount of time for each activity. There is a catalog of activities in the SOW generation automation project or a list of services can be found at our offerings framework page.

Iterate / Review Engagement Estimate

After the first iteration of building detail into the straw-man, the Engagement manager posts a request for review in the professional services slack channel to the Account team. Often the Engagement Manager will get on a zoom call with the customer and provide context and gather feedback from the customer.

Generate SOW

Once the Engagement Manager can get buy in from the account team and/or customer on the size and activities included in the services engagement, the SOW generation automation can be run using the straw-man as an input. The engagement manager is responsible for running this automation which can be done by following the instructions on the readme in the above project.

Iterate/Review SOW Data

Once the SOW is generated, it is ready for review by the account team. Iteration can occur here but should be minimized if the proper iteration was done on the Straw-man Steps.

SOW Review/Approval

After one or more rounds of iteration on feedback from the account team, the SOW will be ready for Review and Approval by Sr. Director of Professional Services. The review processes are signaled in the professional services slack channel and at-mention the GitLab engagement stakeholders.

3. Delivery Prep

- DRI: Project Manager
- Supported By: EM, PM, PSE, TA, Account Team

Once the engagement moves to stage 6, awaiting signature in salesforce, the engagement manager schedules an intro meeting between the PM and the customer stakeholder to discuss "how we manage projects at GitLab". During this call, the PM will also discuss resource assignment, and scheduling details. Simultaneously, the PM will schedule an internal transition meeting with the EM, PM, PSE, TA, and Account Team to review the technical discussions that occurred during the customer scoping meetings. This is a opportunity for the entire team to discuss and disclose the inner workings and dynamics of the customer project team.

PM Introduction

Engagement managers introduce the PS Project Manager as they are in contract with the customer during the scoping exercise and are perfectly positioned to do so.

Resource Identification

The PS Project Coordinator, working closely with the Regional Delivery Manager, will identify the best resource to deliver the project based on skillset, timezone, personality, tenure, and availability.

Project Prep call

This is a call between the customer stakeholder and the PS Project Manager to discuss HOW we work. Where we track project progress, what status report will look like, what the meeting cadence might be, a review of the contract terms that affect how we work such as minimum hours per week, expiration dates, delay language, and most importantly, how to achieve mutual agreement with the customer on a preliminary project schedule, prior to the kick-off call which includes the entire project team.

Check out the PS Delivery methodology to understand the details around pre-sales handoff to the delivery team, to ensure a successful relationship and productive engagement.

4. SOW Execution/Close

- DRI Engagement Manager
- Supported By: PS Leader, Sr. PSE, SAE/AE, Legal, Finance, Deal Desk

Finance Approval

The engagement manager can kick off the finance approval process in Salesforce. TODO: Add more details.

Legal Approval

The Engagement Manager is responsible for creating a Legal Approval Case in Salesforce from the associated opportunity. This process often involves reviewing and accepting redlines from the customer and from our legal team. The source of truth for the latest SOW is managed in the SFDC legal case. The EM should coordinate who "holds the pen" to ensure we maintain version control of the SOW with the latest redlines.

Customer Signature

Once the SOW has been approved by PS leadership, Legal and Revenue, the account team owns the process of executing the SOW. They should take the approved SOW from the legal case and route it for signature with the customer.

Deal Desk updates SFDC

Once the SOW is fully executed, the deal desk team updates the Salesforce PS-Only opportunity to closed-won.

Journal Epic (Delivery Kit)

TODO: update this with documents and delivery kits that are organized to help the delivery team be successful with the engagement.

-->

<!--This page provides scoping questions designed to collect details and uncover the customer's required capabilities. This ensures alignment of the SOW and PS delivery with the positive business outcomes the customer is looking for. While not an exhaustive list, these questions and suggestions will help spark the discovery conversations.

Migration scoping questions

Migrations are one of the most complex types of services in any technical field. Systems store data in a variety of ways that evolves. Also, customers and users often use the same data model and system to represent completely different logical units to their teams. To ensure a transition that meets the customer's needs, we want to make sure we understand their usage of their current systems.

GitLab to GitLab scoping questions

The following questions are about bringing multiple GitLab instances together into a single "parent" instance:

1. How many instances need to be migrated together?
1. Breakdown of repositories per instance (and total git storage space)
1. Breakdown of users per instance
1. Breakdown of groups per instance
1. Breakdown of disk space per instance
1. Do any of the users use SSO (LDAP, SAML, etc.) to log in users?
1. What is the version of every GitLab instance involved?
1. What integrations are used on each instance?

1. For each instance, how is artifact, upload, container registry and Git LFS storage handled?

SVN to Git scoping questions

SVN to Git Questions:

1. What is the structure of the SVN repos and subprojects? Do they follow the "standard" of:

html

Repository

Project 1

branches/trunks/tags

Project 2

branches/trunks/tags

1. Any other variations? Example:

html

Trunk

Project 1

Project 2

Tags

tag name

Project 1

Project 2

Branches

Branch name

Project 1

Project 2

1. How many SVN repositories are there? How are they broken up?

1. What is the overall size of the SVN repos?

1. How much history (e.g. tags, branches, etc.) should be migrated?

1. Are any binary files stored in SVN?

1. How are you currently using SVN externals? Do you have some example use cases?

1. Which migration/conversion environment (local, VM, cloud) and OS (Windows, Linux, macOS) is preferred?

GitLab self-managed to GitLab.com scoping questions

Self-Manged to GitLab.com questions:

1. How many repositories are there?

1. What is the average size of the repositories? What is the size of the largest repositories?

1. How many groups are there?

1. How many users?

1. Do we want the same structure of groups/projects/user access? Do we want it to be restructured?

1. Do you use other tools with GitLab. If so, which ones?

1. How are the runners currently set up?

1. Where are they hosted?

1. How many specific runners do you have and what are the details of these runners?

1. Are you looking to make any changes to the runner strategy at your organization or will it stay the same?

1. GitLab.com uses SAML for authorization. What SAML tool would your organization be using?

Implementation scoping questions

For scoping infrastructure implementation, we have several questions that can help us understand the scope. To get started, make a copy of the spreadsheet below and fill in the answers. If there are questions you want to add, ping the @ps-team in Slack channel professional-services. Then you can attach this document to the issue created by the GitLab Services (SoW) Calculator.

Implementation Infrastructure Scoping Questions

Note: This document contains tabs for various public and private cloud providers - AWS, GCP, Azure, OpenShift and other on-prem deployments - you should only have to fill out the tab(s) relative to your customer.-->

<!--

title: "Post-Sales"

description: "Describes the workflow governing delivery of GitLab professional service projects."

Overview

The purpose of this page is to describe the workflow governing delivery of professional service projects. We will start by outlining the general workflow that is common across all service delivery categories. Then we will describe the differences for engagements that do not fit the general workflow shown below.

Source, internal only

Update this below

GitLab professional services employs three different workflows that control projects for the following categories: standard professional services, education, and dedicated engineers. Each of these categories have unique attributes that warrant a different approach.

1. Pre-sales

Note: for presales scoping and SOW signature, see the pre-sales methodology page

PS/EM Assessment

TODO: Add content about what happens in this step

2. PS Project Planning

Resource Assignment

Resource assignment happens only after the SOW is received with Customer signature. The Sr. PS Operations team identifies delivery resources based on the needs of the engagement.

Sales to PS Handoff

The Project Coordinator will schedule a handoff call with the account team (SAE/AE, SA, CSM), the EM who scoped the engagement, and the delivery team (PM, PSE, Trainer, Training Coordinator). The meeting will start with the EM and account team describing the customers high-level goals, current state, and desired business outcomes. The Account team should note what the growth potential is for this customer and their strategic plan to drive that growth. These topics give the context about the customer to the delivery team to understand where there might be follow on opportunities.

The EM will discuss in more detail the details of the engagement. The Delivery team will ask clarifying questions for what is in scope and what is out of scope. The PMs will review where the project definition document is stored, which is usually in the Active Projects folder under the Customer project.

TODO: Add or link to content about what happens in this step. E.g. List link to the skills matrix, Show example of skills requirements breakdown that come from